

# MCP Security with Permissions and Elicitation

**Estimated time needed:** 60 minutes

In this lab, you'll build an advanced MCP security system featuring **permissions** for policy-based access control and **elicitation** for structured user input. You'll create a **base permission-aware client** that enforces security policies, then build two applications on top of it: a **GUI client app** with permission management interface and an **AI-powered host app** that assesses risk and requests user approval for sensitive operations.

This design demonstrates enterprise-grade security patterns for production MCP deployments with comprehensive audit logging.

## Learning Objectives

After completing this lab, you will be able to:

- Implement permission policies (allow, deny, ask) for tool execution
- Configure argument-specific permission rules for granular control
- Use elicitation to request structured user input with JSON schemas
- Build audit logging systems for compliance and security monitoring
- Assess operation risk levels and apply appropriate controls
- Create interactive approval workflows for sensitive operations
- Manage permissions programmatically and through GUI interfaces

## Prerequisites

Before starting this lab, you should have:

- Experience building MCP clients with base/derived class architecture
- Understanding of MCP protocol methods (tools, resources, prompts)

- Basic knowledge of object-oriented programming (inheritance)
- Understanding of access control and security policies
- Familiarity with JSON Schema validation
- Awareness of audit logging and compliance requirements

# Lab Setup

Let's set up your development environment for MCP security features.

## Create Virtual Environment

bash

```
python3.11 -m venv mcp_security_env  
source mcp_security_env/bin/activate
```

Run

## Install Dependencies

Install the MCP SDK, FastMCP, Gradio, and OpenAI:

bash

```
pip install mcp==1.16.0 fastmcp==2.12.5 gradio==5.49.1 openai==2.6.1
```

Run

## Create Project Directory

bash

```
mkdir mcp_security_lab  
cd mcp_security_lab
```

Run

## Create Data Directory

Create a directory for storing permissions and audit logs:



bash



```
mkdir data
```

▶ Run

This directory will contain:

- `permissions.json` - Permission policies for tools
- `audit.log` - Audit trail of all operations

You're now ready to build secure MCP applications with permissions and elicitation!

# Build the Permission-Aware MCP Server

Let's build an MCP server that provides tools at different risk levels and uses elicitation to request user approval for sensitive operations.

Open `mcp_permission_server.py` in IDE

## Step 1: Import Dependencies and Setup

```
import json
from pathlib import Path
from datetime import datetime
from fastmcp import FastMCP
import warnings

# Suppress deprecation warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

BASE_DIR = Path(__file__).parent / "data"
BASE_DIR.mkdir(exist_ok=True)

mcp = FastMCP("Permission-Aware MCP Server")
```

What this does:

- Imports FastMCP for building the server
- Suppresses deprecation warnings for cleaner output
- Sets up a data directory for audit logs and permissions
- Creates the MCP server instance
- Ensures the data directory exists

## Step 2: Add Low-Risk Tool (`read_file`)

Add a safe read-only operation:



python



```
@mcp.tool()
def read_file(filepath: str) -> str:
    """
    Read a file from the data directory. (Risk: LOW)

    Args:
        filepath: Path to the file relative to data directory
    """
    try:
        file_path = BASE_DIR / filepath

        if not file_path.exists():
            return f"Error: File {filepath} not found"

        return file_path.read_text()
    except Exception as e:
        return f"Error reading file: {str(e)}"
```

What this does:

- Provides safe read-only file access
- Risk level: LOW (no data modification)
- No elicitation needed for read operations
- Returns error messages for invalid paths

### Step 3: Add Medium-Risk Tool (write\_file)

Add a file writing operation that should require confirmation:



python



```
@mcp.tool()
def write_file(filepath: str, content: str) -> str:
    """
    Write content to a file in the data directory. (Risk: MEDIUM)

    This operation modifies data and should require user confirmation via

    Args:
        filepath: Path to the file relative to data directory
        content: Content to write to the file
    """
    try:
        file_path = BASE_DIR / filepath
        file_path.write_text(content)

        # Log the operation
        log_entry = f"[{datetime.now().isoformat()}] WRITE: {filepath}\n"
        audit_log = BASE_DIR / "audit.log"
        with open(audit_log, "a") as f:
            f.write(log_entry)

        return f"Successfully wrote to {filepath}"
    except Exception as e:
        return f"Error writing file: {str(e)}"
```

What this does:

- Writes content to files (data modification)
- Risk level: MEDIUM (modifies existing data)
- Should trigger elicitation for user confirmation
- Logs all write operations to audit log

## Step 4: Add High-Risk Tool (delete\_file)

Add a file deletion operation that requires detailed confirmation:



python



```
@mcp.tool()
def delete_file(filepath: str) -> str:
    """
    Delete a file from the data directory. (Risk: HIGH)

    This operation is destructive and should require detailed user confirm

    Args:
        filepath: Path to the file to delete
    """
    try:
        file_path = BASE_DIR / filepath

        if not file_path.exists():
            return f"Error: File {filepath} not found"

        file_path.unlink()

        # Log the operation
        log_entry = f"[{datetime.now().isoformat()}] DELETE: {filepath}\n"
        audit_log = BASE_DIR / "audit.log"
        with open(audit_log, "a") as f:
            f.write(log_entry)

        return f"Successfully deleted {filepath}"
    except Exception as e:
        return f"Error deleting file: {str(e)}"
```

What this does:

- Deletes files (irreversible operation)
- Risk level: HIGH (data loss potential)
- Should require detailed elicitation with reason
- Logs all delete operations for audit trail

## Step 5: Add Critical-Risk Tool (execute\_command)

Add a system command execution operation (for demonstration):





python



```
@mcp.tool()
def execute_command(command: str) -> str:
    """
    Execute a system command. (Risk: CRITICAL)

    This operation can affect system state and should require extensive us
    For security, this is a simulation only.

    Args:
        command: The command to execute (simulated)
    """
    # Simulate command execution without actually running it
    log_entry = f"[{datetime.now().isoformat()}] EXECUTE (simulated): {com
    audit_log = BASE_DIR / "audit.log"
    with open(audit_log, "a") as f:
        f.write(log_entry)

    return f"Simulated execution of command: {command}\n(Actual execution
```

What this does:

- Simulates system command execution (demo only)
- Risk level: CRITICAL (could affect entire system)
- Should require extensive elicitation with purpose and risk acknowledgment
- Logs attempted executions for security monitoring

## Step 6: Add Resources for Audit and Configuration

Add resources to expose audit logs and permission configuration:



python



```
@mcp.resource("file://audit/log")
def get_audit_log() -> str:
    """Get the audit log of all operations."""
    audit_log = BASE_DIR / "audit.log"
    if not audit_log.exists():
        return "No audit log entries yet."
    return audit_log.read_text()

@mcp.resource("file://config/permissions")
def get_permissions_config() -> str:
    """Get the current permissions configuration."""
    permissions_file = BASE_DIR / "permissions.json"
    if not permissions_file.exists():
        return json.dumps({
            "read_file": "allow",
            "write_file": "ask",
            "delete_file": "deny",
            "execute_command": "deny"
        }, indent=2)
    return permissions_file.read_text()
```

What this does:

- Exposes audit log as a readable resource
- Exposes permissions configuration for inspection
- Provides default permissions if none exist
- Enables monitoring and debugging of security policies

## Step 7: Add Security Review Prompt

Add a prompt template for security analysis:

<>

python

```
@mcp.prompt()
def security_review(operation: str, risk_level: str) -> list[dict]:
    """
    Generate a security review prompt for an operation.

    Args:
        operation: The operation to review
        risk_level: The risk level (LOW, MEDIUM, HIGH, CRITICAL)
    """
    return [
        {
            "role": "user",
            "content": f"""Review this operation for security implications

Operation: {operation}
Risk Level: {risk_level}

Please analyze:
1. What data or systems could be affected?
2. What are the potential security risks?
3. What safeguards should be in place?
4. Should this operation require user approval?
5. What should be logged for audit purposes?
"""
        }
    ]
```

What this does:

- Creates a template for security analysis
- Takes operation details and risk level as parameters
- Generates structured security review questions
- Can be used by LLMs or security teams for assessment

## Step 8: Add Server Entry Point

Add the main entry point to run the server:

<>

python

```
if __name__ == "__main__":
    mcp.run(transports="stdio")
```

What this does:

- Runs the MCP server using STDIO transport
- Server will communicate via standard input/output
- Can be launched as a subprocess by clients
- Ready for local process communication

Click below to see the complete code for `mcp_permission_server.py`:

► Complete code for `mcp_permission_server.py` (click to expand)

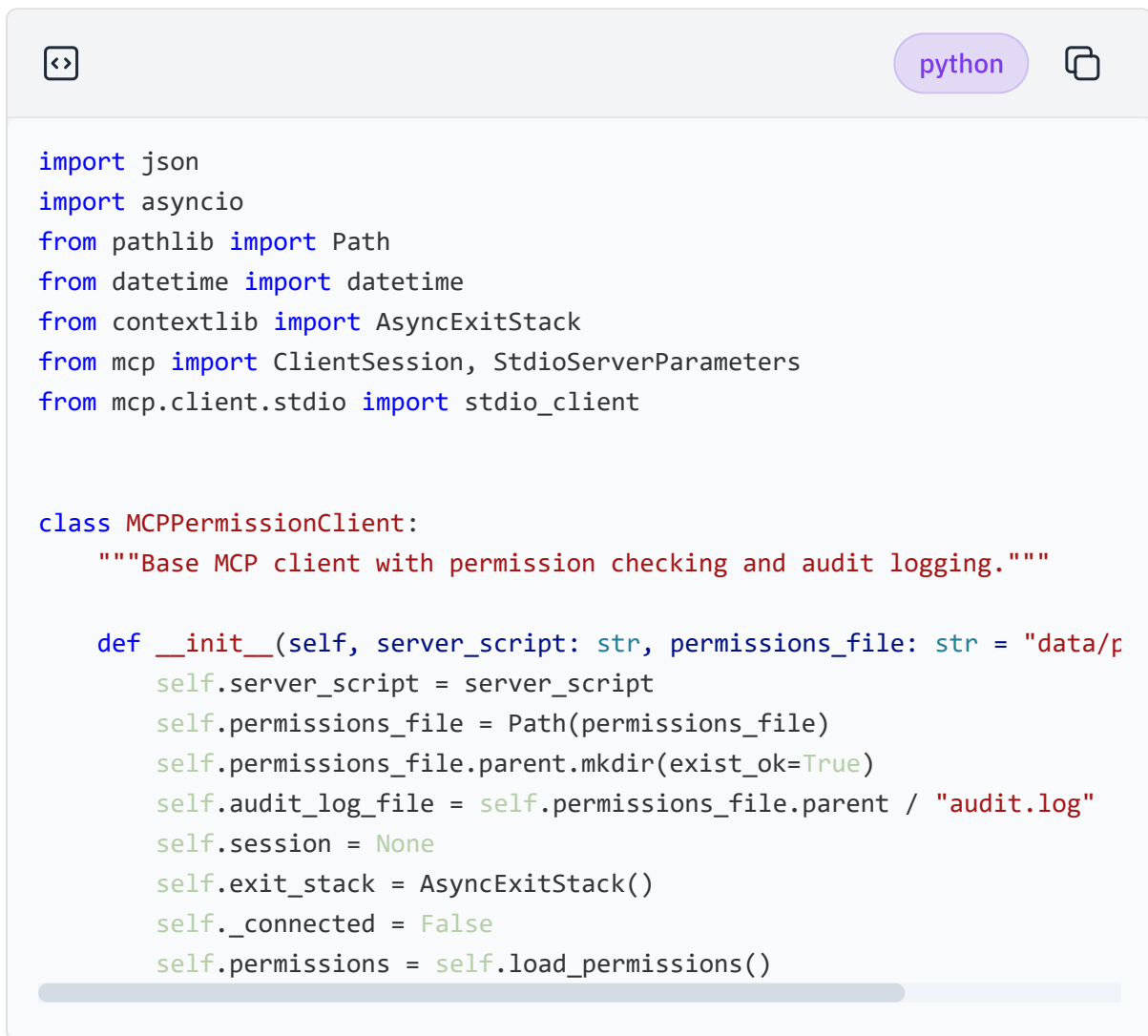
# Build the Base Permission Client

Now let's build a base client that enforces permissions, handles elicitation, and maintains audit logs. This client will be inherited by both the GUI and AI host applications.

Open `mcp_permission_client_base.py` in IDE

## Step 1: Import Dependencies and Create the Class

Create `mcp_permission_client_base.py` and start with imports and class definition:



```
import json
import asyncio
from pathlib import Path
from datetime import datetime
from contextlib import AsyncExitStack
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

class MCPPermissionClient:
    """Base MCP client with permission checking and audit logging."""

    def __init__(self, server_script: str, permissions_file: str = "data/p
        self.server_script = server_script
        self.permissions_file = Path(permissions_file)
        self.permissions_file.parent.mkdir(exist_ok=True)
        self.audit_log_file = self.permissions_file.parent / "audit.log"
        self.session = None
        self.exit_stack = AsyncExitStack()
        self._connected = False
        self.permissions = self.load_permissions()
```

What this does:

- Takes server script path and permissions file location
- Creates data directory if it doesn't exist

- Initializes session management with AsyncExitStack
- Uses `_connected` flag for lazy initialization
- Loads existing permissions from file

## Step 2: Implement Connection Management

Add the connect method for lazy initialization:

A code editor window with a light gray header bar. On the left is a code icon, on the right is a 'python' language selector and a copy icon. The main area contains Python code for an async connect method. The code checks if already connected, sets up server parameters for 'python', creates a stdio transport, initializes a client session, and sets the \_connected flag to True.

```
async def connect(self):  
    """Connect to the MCP server via STDIO. Safe to call multiple time  
    if self._connected:  
        return  
  
    server_params = StdioServerParameters(  
        command="python",  
        args=[self.server_script],  
        env=None  
    )  
  
    stdio_transport = await self.exit_stack.enter_async_context(  
        stdio_client(server_params)  
    )  
  
    read, write = stdio_transport  
    self.session = await self.exit_stack.enter_async_context(  
        ClientSession(read, write)  
    )  
  
    await self.session.initialize()  
    self._connected = True
```

What this does:

- Only connects once (idempotent)
- Launches server as subprocess with STDIO transport
- Creates client session for communication
- Initializes the MCP protocol
- Sets connected flag to prevent duplicate connections

## Step 3: Permission Management Methods

Add methods to load, save, and check permissions:

```
def load_permissions(self) -> dict:
    """Load permissions from file or return defaults."""
    if self.permissions_file.exists():
        return json.loads(self.permissions_file.read_text())
    return {
        "read_file": "allow",
        "write_file": "ask",
        "delete_file": "deny",
        "execute_command": "deny"
    }

def save_permissions(self):
    """Save current permissions to file."""
    self.permissions_file.write_text(json.dumps(self.permissions, inde

def check_permission(self, tool_name: str, arguments: dict) -> str:
    """
    Check permission for a tool call.

    Returns: "allow", "deny", or "ask"
    """
    # Check for argument-specific permission
    arg_key = f"{tool_name}:{json.dumps(arguments, sort_keys=True)}"
    if arg_key in self.permissions:
        return self.permissions[arg_key]

    # Check for general tool permission
    return self.permissions.get(tool_name, "ask")
```

What this does:

- **load\_permissions** : Reads from file or provides secure defaults
- **save\_permissions** : Persists permissions to JSON file
- **check\_permission** : Returns policy for a specific tool/arguments combination
- Supports argument-specific permissions for granular control
- Defaults to "ask" for unknown tools (secure by default)

## Step 4: Audit Logging Method

Add audit logging functionality:

```
def log_audit(self, operation: str, decision: str, reason: str = ""):
    """Log an operation to the audit log."""
    timestamp = datetime.now().isoformat()
    log_entry = f"[{timestamp}] {operation} - Decision: {decision}"
    if reason:
        log_entry += f" - Reason: {reason}"
    log_entry += "\n"

    with open(self.audit_log_file, "a") as f:
        f.write(log_entry)
```

What this does:

- Creates timestamped audit log entries
- Records operation, decision (allow/deny/ask), and reason
- Appends to persistent audit log file
- Provides compliance and security monitoring trail

## Step 5: Elicitation Request Method (Conceptual)

Add a method for requesting structured user input:





python



```
async def request_elicitation(self, schema: dict, description: str) ->
    """
    Request structured user input via elicitation.

    This is a conceptual implementation. In production, this would
    trigger a UI dialog and wait for user response.

    Args:
        schema: JSON schema for the required input
        description: Human-readable description of what's needed

    Returns:
        Dictionary with user's input
    """
    # Conceptual: In real implementation, this would show a UI dialog
    # and block until user provides input or cancels
    print(f"\nElicitation requested: {description}")
    print(f"Schema: {json.dumps(schema, indent=2)}")
    print("(Conceptual - automatic approval for demo)")

    # For demo purposes, return empty dict (representing user approval
    return {}
```

What this does:

- Defines the interface for elicitation
- Takes JSON schema and description
- In production, would show UI dialog and wait for user input
- For demo, logs the request and auto-approves
- Returns user's structured input

## Step 6: Protocol Methods - Tools

Add methods for working with MCP tools:



python



```
async def list_tools(self):
    """List all available tools from the server."""
    await self.connect()
    result = await self.session.list_tools()
    return result.tools

async def call_tool_with_permission(self, tool_name: str, arguments: dict):
    """Call a tool after checking permissions."""
    await self.connect()

    if arguments is None:
        arguments = {}

    # Check permission
    permission = self.check_permission(tool_name, arguments)

    if permission == "deny":
        self.log_audit(f"TOOL: {tool_name}", "DENIED", "Policy: deny")
        return [type('obj', (), {'text': f"Permission denied for tool: {tool_name}"})]

    if permission == "ask" and not approved:
        # Log that approval was requested
        self.log_audit(f"TOOL: {tool_name}", "ASK", "Awaiting approval")

        # Return approval request message
        approval_msg = f"""Permission required for tool: {tool_name}
Arguments: {json.dumps(arguments, indent=2)}

This tool requires approval before execution.
Please approve this operation in the GUI to proceed."""
        return [type('obj', (), {'text': approval_msg})]

    # Execute the tool
    self.log_audit(f"TOOL: {tool_name}", "ALLOWED", f"Policy: {permission}")
    result = await self.session.call_tool(tool_name, arguments=arguments)
    return result.content
```

What this does:

- Lists all tools from the server
- Checks permission before execution
- Denies if policy is "deny"
- Requests approval if policy is "ask"

- Executes tool only after approval

## Step 7: Protocol Methods - Resources

Add methods for working with MCP resources:

```
async def list_resources(self):
    """List all available resources from the server."""
    await self.connect()
    result = await self.session.list_resources()
    return result.resources

async def read_resource(self, uri: str):
    """Read a resource by URI."""
    await self.connect()
    result = await self.session.read_resource(uri=uri)
    return result.contents
```

What this does:

- Lists available resources with their URIs
- Reads resource content by URI
- No permission checks for resources (read-only)
- Returns structured resource data

## Step 8: Protocol Methods - Prompts

Add methods for working with MCP prompts:

python

```
async def list_prompts(self):
    """List all available prompts from the server."""
    await self.connect()
    result = await self.session.list_prompts()
    return result.prompts

async def get_prompt(self, prompt_name: str, arguments: dict = None):
    """Get a rendered prompt template."""
    await self.connect()
    if arguments is None:
        arguments = {}
    result = await self.session.get_prompt(name=prompt_name, arguments=arguments)
    return result.messages
```

What this does:

- Lists available prompt templates with their parameters
- Renders prompts with provided arguments
- No permission checks for prompts (read-only)
- Returns formatted messages

## Step 9: Cleanup Method

Add cleanup for proper resource management:

python

```
async def cleanup(self):
    """Clean up resources."""
    await self.exit_stack.aclose()
```

What this does:

- Closes all async context managers
- Properly terminates server subprocess
- Releases system resources
- Should be called when done with client

► Complete code for `mcp_permission_client_base.py` (click to expand)

# Build the GUI Client App

Now let's build a GUI application with an interactive interface for managing permissions, viewing audit logs, and testing tools with permission enforcement.

Open `mcp_permission_client_app.py` in IDE

## Step 1: Import Dependencies and Create the Class

Create `mcp_permission_client_app.py` and start with imports and class definition:

```
<> python   
  
import sys  
import json  
import gradio as gr  
from mcp_permission_client_base import MCPPermissionClient  
  
class MCPPermissionClientApp(MCPPermissionClient):  
    """GUI client application with permission management interface."""  
  
    def __init__(self, server_script: str):  
        super().__init__(server_script)  
        self.tools_cache = []  
        self.prompts_cache = []
```

What this does:

- Inherits all protocol and permission methods from base client
- Adds caches for tools and prompts to populate dropdowns
- No need to reimplement connection, permissions, or audit logging
- Focuses solely on GUI presentation layer

## Step 2: GUI Method - List Tools with Permission Status

Add a method to list tools with their permission status:



python



```
async def gui_list_tools(self):
    """List tools with their permission status for GUI."""
    await self.connect()
    tools = await self.list_tools()

    output = "Available tools:\n\n"
    self.tools_cache = []

    for tool in tools:
        tool_name = tool.name
        permission = self.permissions.get(tool_name, "ask")
        self.tools_cache.append(tool_name)

        output += f"- {tool_name}\n"
        output += f"  Permission: {permission.upper()}\n"
        if tool.description:
            output += f"  Description: {tool.description}\n"
        output += "\n"

    choices = [f"{name} ({self.permissions.get(name, 'ask')})" for name in self.tools_cache]
    return output, gr.update(choices=choices)
```

What this does:

- Lists all tools from the server
- Shows current permission status for each tool
- Updates tools cache for dropdown population
- Returns formatted output and updated dropdown choices
- Displays permission policy inline with tool names

### Step 3: GUI Method - Call Tool with Permission Check

Add a method to call tools through the GUI:



python



```
async def gui_call_tool(self, tool_selection: str, arguments_json: str
    """Call a tool with permission checking for GUI."""
    if not tool_selection:
        return "Please select a tool first"

    # Extract tool name from selection (format: "tool_name (permission
    tool_name = tool_selection.split(" ")[0]

    # Parse arguments
    try:
        arguments = json.loads(arguments_json) if arguments_json.strip
    except json.JSONDecodeError as e:
        return f"Invalid JSON in arguments: {str(e)}"

    # Call tool with permission check
    result = await self.call_tool_with_permission(tool_name, arguments

    # Extract text content
    if isinstance(result, list) and len(result) > 0:
        content = result[0]
        if hasattr(content, 'text'):
            return content.text
        return str(content)

    return str(result)
```

What this does:

- Extracts tool name from dropdown selection
- Parses JSON arguments with error handling
- Uses inherited permission checking from base client
- Logs to audit trail automatically
- Returns formatted result or error message

## Step 4: GUI Method - List Resources

Add a method to list available resources:





python



```
async def gui_list_resources(self):
    """List resources for GUI."""
    await self.connect()
    resources = await self.list_resources()

    output = "Available resources:\n\n"
    for resource in resources:
        output += f"- {resource.uri}\n"
        if resource.name:
            output += f"  Name: {resource.name}\n"
        if resource.description:
            output += f"  Description: {resource.description}\n"
        output += "\n"

    return output
```

What this does:

- Lists all resources from the server
- Shows URIs, names, and descriptions
- No permission checks needed (resources are read-only)
- Returns formatted list for display

## Step 5: GUI Method - Read Resource

Add a method to read resource content:



python



```
async def gui_read_resource(self, uri: str):
    """Read a resource for GUI."""
    if not uri.strip():
        return "Please enter a resource URI"

    await self.connect()
    contents = await self.read_resource(uri)

    if isinstance(contents, list) and len(contents) > 0:
        content = contents[0]
        if hasattr(content, 'text'):
            return content.text
        return str(content)

    return str(contents)
```

What this does:

- Reads resource by URI
- Extracts text content from response
- Returns formatted content or error
- No permission enforcement for read-only resources

## Step 6: GUI Method - List and Get Prompts

Add methods for working with prompts:



python



```
async def gui_list_prompts(self):
    """List prompts for GUI."""
    await self.connect()
    prompts = await self.list_prompts()

    output = "Available prompts:\n\n"
    self.prompts_cache = []

    for prompt in prompts:
        self.prompts_cache.append(prompt.name)
        output += f"- {prompt.name}\n"
        if prompt.description:
            output += f"  Description: {prompt.description}\n"
        if hasattr(prompt, 'arguments') and prompt.arguments:
            args = [arg.name for arg in prompt.arguments]
            output += f"  Arguments: {' '.join(args)}\n"
        output += "\n"

    return output, gr.update(choices=self.prompts_cache)

async def gui_get_prompt(self, prompt_name: str, arguments_json: str):
    """Get a rendered prompt for GUI."""
    if not prompt_name:
        return "Please select a prompt first"

    # Parse arguments
    try:
        arguments = json.loads(arguments_json) if arguments_json.strip()
    except json.JSONDecodeError as e:
        return f"Invalid JSON in arguments: {str(e)}"

    # Get prompt
    messages = await self.get_prompt(prompt_name, arguments)

    output = f"Prompt: {prompt_name}\n\n"
    for msg in messages:
        role = getattr(msg, 'role', 'unknown')
        content = getattr(msg, 'content', '')
        if hasattr(content, 'text'):
            content = content.text
        output += f"[{role}]: {content}\n\n"

    return output
```

What this does:

- Renders prompts with provided arguments
- Parses JSON arguments with error handling
- Formats messages for display

## Step 7: GUI Method - Configure Permissions

Add a method to update tool permissions:

```
async def gui_configure_permission(self, tool_name: str, policy: str):  
    """Configure permission for a tool."""  
    if not tool_name:  
        return "Please enter a tool name"  
  
    if policy not in ["allow", "deny", "ask"]:  
        return "Policy must be: allow, deny, or ask"  
  
    self.permissions[tool_name] = policy  
    self.save_permissions()  
  
    return f"Permission updated: {tool_name} = {policy}\nPermissions s
```

What this does:

- Updates permission policy for a specific tool
- Validates policy is one of the three allowed values
- Saves to persistent storage
- Returns confirmation message

## Step 8: GUI Method - View Audit Log

Add a method to display the audit log:

```
async def gui_view_audit_log(self):  
    """View the audit log."""  
    if not self.audit_log_file.exists():  
        return "No audit log entries yet."
```

What this does:

- Reads the audit log file
- Shows all logged operations with timestamps
- Returns empty message if no log exists
- Provides compliance and security monitoring

## Step 9: Create the Gradio Interface

Create the main interface with four tabs:



python



```
def create_interface(self):
    """Create the Gradio interface with permission management."""

    with gr.Blocks(title="MCP Permission Client") as interface:
        gr.Markdown("""
        # MCP Permission Client
        Manage permissions, view audit logs, and interact with MCP too
        """)

        with gr.Tabs():
            with gr.Tab("Tools"):
                gr.Markdown("### List and Call Tools with Permission E
                with gr.Row():
                    with gr.Column():
                        list_tools_btn = gr.Button("List Tools", varia
                        tools_output = gr.Textbox(label="Available Toc

                    with gr.Column():
                        tool_dropdown = gr.Dropdown(label="Select Tool
                        tool_args = gr.Textbox(
                            label="Arguments (JSON)",
                            placeholder='{"filepath": "test.txt"}',
                            lines=3
                        )
                    with gr.Row():
                        call_tool_btn = gr.Button("Call Tool", var
                        approve_tool_btn = gr.Button("Approve & Ex
                        tool_result = gr.Textbox(label="Result", lines

                list_tools_btn.click(
                    fn=self.gui_list_tools,
                    outputs=[tools_output, tool_dropdown]
                )

                call_tool_btn.click(
                    fn=self.gui_call_tool,
                    inputs=[tool_dropdown, tool_args],
                    outputs=tool_result
                )

            async def gui_approve_tool(tool_selection, arguments_j
                return await self.gui_call_tool(tool_selection, ar

            approve_tool_btn.click(
                fn=gui_approve_tool,
                inputs=[tool_dropdown, tool_args],
                outputs=tool_result
            )
```

```

gr.Markdown("### List and Read Resources")
with gr.Row():
    with gr.Column():
        list_resources_btn = gr.Button("List Resources")
        resources_output = gr.Textbox(label="Available Resources")

    with gr.Column():
        resource_uri = gr.Textbox(
            label="Resource URI",
            placeholder="file:///audit/log"
        )
        read_resource_btn = gr.Button("Read Resource")
        resource_content = gr.Textbox(label="Resource Content")

list_resources_btn.click(
    fn=self.gui_list_resources,
    outputs=resources_output
)

read_resource_btn.click(
    fn=self.gui_read_resource,
    inputs=resource_uri,
    outputs=resource_content
)

with gr.Tab("Prompts"):
    gr.Markdown("### List and Get Prompts")
    with gr.Row():
        with gr.Column():
            list_prompts_btn = gr.Button("List Prompts", variant="primary")
            prompts_output = gr.Textbox(label="Available Prompts")

        with gr.Column():
            prompt_dropdown = gr.Dropdown(label="Select Prompt")
            prompt_args = gr.Textbox(
                label="Arguments (JSON)",
                placeholder='{"operation": "write_file", "args": {}}',
                lines=2
            )
            get_prompt_btn = gr.Button("Get Prompt", variant="primary")
            prompt_result = gr.Textbox(label="Prompt Result")

list_prompts_btn.click(
    fn=self.gui_list_prompts,
    outputs=[prompts_output, prompt_dropdown]
)

get_prompt_btn.click(
    fn=self.gui_get_prompt,
    inputs=[prompt_dropdown, prompt_args],
    outputs=prompt_result
)

```

```

with gr.Tab("Permissions"):
    gr.Markdown("### Manage Permissions and View Audit Log")
    with gr.Row():
        with gr.Column():
            gr.Markdown("***Configure Tool Permission**")
            list_tools_for_perm_btn = gr.Button("Load Tool")
            perm_tool_name = gr.Dropdown(
                label="Tool Name",
                choices=[],
                allow_custom_value=True
            )
            perm_policy = gr.Radio(
                choices=["allow", "deny", "ask"],
                label="Permission Policy",
                value="ask"
            )
            save_perm_btn = gr.Button("Save Permission", v
            perm_result = gr.Textbox(label="Result", lines

        with gr.Column():
            gr.Markdown("***Audit Log**")
            view_audit_btn = gr.Button("View Audit Log", v
            audit_output = gr.Textbox(label="Audit Log", l

    async def load_tools_for_dropdown():
        tools = await self.list_tools()
        tool_names = [tool.name for tool in tools]
        return gr.Dropdown(choices=tool_names)

    list_tools_for_perm_btn.click(
        fn=load_tools_for_dropdown,
        outputs=perm_tool_name
    )

    save_perm_btn.click(
        fn=self.gui_configure_permission,
        inputs=[perm_tool_name, perm_policy],
        outputs=perm_result
    )

    view_audit_btn.click(
        fn=self.gui_view_audit_log,
        outputs=audit_output
    )

return interface

```

What this does:

- Creates 4 tabs: Tools, Resources, Prompts, Permissions



- Resources tab: List and read resources
- Prompts tab: List and get prompts
- Permissions tab: Configure policies and view audit log
- Two-column layouts for discovery and usage
- Permission enforcement automatically applied

## Step 10: Add Main Entry Point

Add the main function to run the application:



```
def main():
    if len(sys.argv) < 2:
        print("Usage: python mcp_permission_client_app.py <server_script>")
        print("Example: python mcp_permission_client_app.py mcp_permission")
        sys.exit(1)

    server_script = sys.argv[1]

    client = MCPPPermissionClientApp(server_script)
    interface = client.create_interface()
    interface.queue().launch(server_name="127.0.0.1", server_port=7863)

if __name__ == "__main__":
    main()
```

What this does:

- Accepts server script path as command-line argument
- Creates the permission-aware GUI client
- Launches on port 7863 (distinct from other labs)
- Uses `.queue()` for proper async support

Click below to see the complete code for `mcp_permission_client_app.py`:

► Complete code for `mcp_permission_client_app.py` (click to expand)



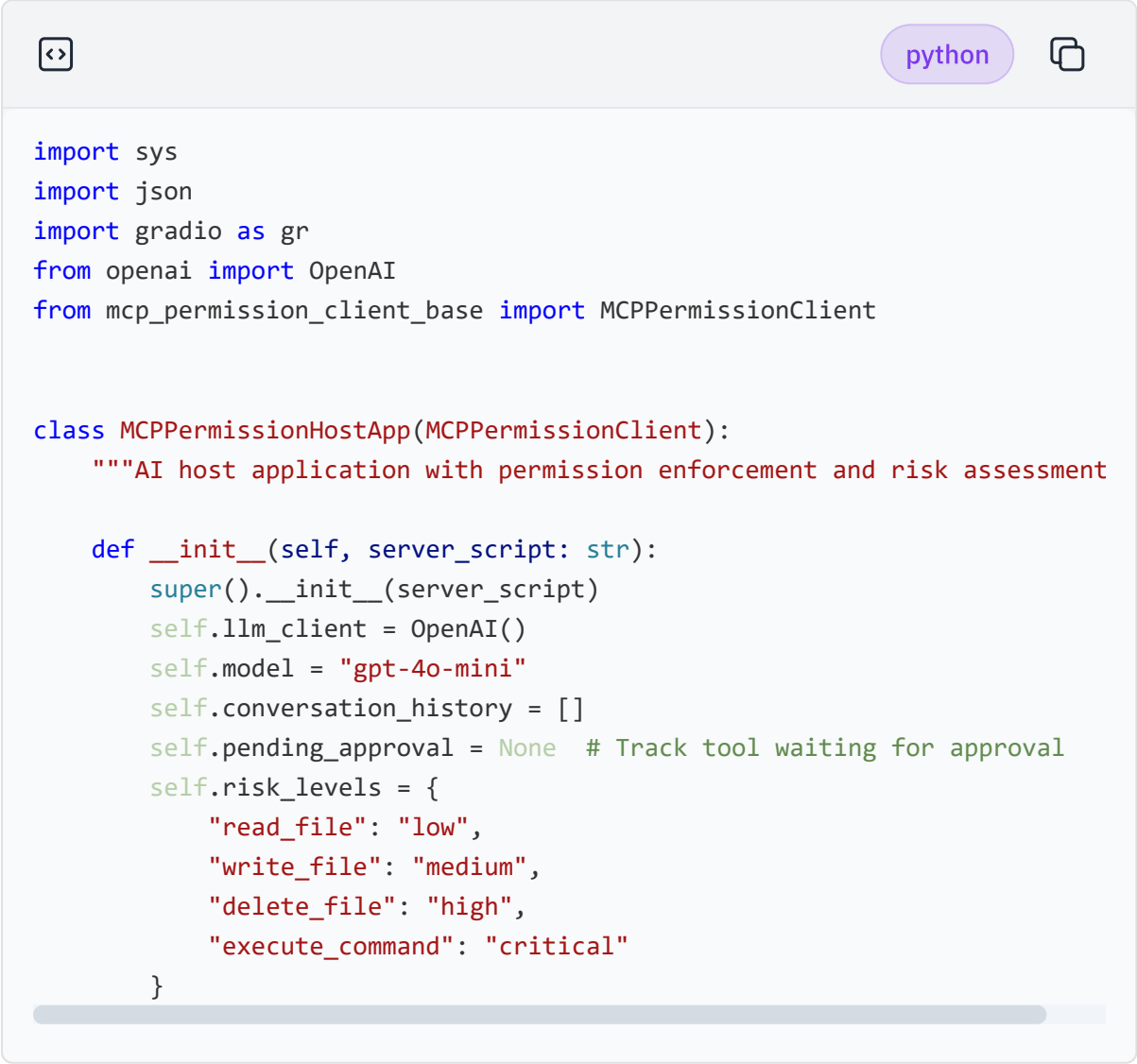
# Build the AI Host App

Now let's build an AI-powered host application that integrates OpenAI's GPT-4o-mini with permission enforcement, risk assessment, and audit logging. This demonstrates how to build secure LLM applications with MCP.

Open `mcp_permission_host_app.py` in IDE

## Step 1: Import Dependencies and Create the Class

Create `mcp_permission_host_app.py` and start with imports and class definition:



```
import sys
import json
import gradio as gr
from openai import OpenAI
from mcp_permission_client_base import MCPPermissionClient

class MCPPermissionHostApp(MCPPermissionClient):
    """AI host application with permission enforcement and risk assessment"""

    def __init__(self, server_script: str):
        super().__init__(server_script)
        self.llm_client = OpenAI()
        self.model = "gpt-4o-mini"
        self.conversation_history = []
        self.pending_approval = None # Track tool waiting for approval
        self.risk_levels = {
            "read_file": "low",
            "write_file": "medium",
            "delete_file": "high",
            "execute_command": "critical"
        }
```

What this does:

- Inherits all permission and protocol methods from base client

- Maintains conversation history for context
- Tracks pending approvals for interactive workflow
- Defines risk levels for each tool
- Reuses permission checking, audit logging, and elicitation

## **Step 2a: Get Available Tools - Real MCP Tools**

Create a method to convert MCP tools into OpenAI function calling format with permission metadata:



python



```
async def get_available_tools(self):
    """Get all available tools in OpenAI function calling format with
    await self.connect()

    # Get real MCP tools
    mcp_tools = await self.list_tools()

    openai_tools = []

    # Add real MCP tools with permission information
    for tool in mcp_tools:
        tool_schema = {
            "type": "function",
            "function": {
                "name": tool.name,
                "description": tool.description or f"Execute {tool.name}"
                "parameters": {
                    "type": "object",
                    "properties": {}
                }
            }
        }

        # Add permission and risk level to description
        permission = self.permissions.get(tool.name, "ask")
        risk = self.risk_levels.get(tool.name, "medium")
        tool_schema["function"]["description"] += f" (Permission: {permission}, Risk: {risk})"

        # Convert MCP input schema to OpenAI parameters
        if hasattr(tool, 'inputSchema') and tool.inputSchema:
            schema = tool.inputSchema
            if isinstance(schema, dict):
                if "properties" in schema:
                    tool_schema["function"]["parameters"]["properties"] = schema["properties"]
                if "required" in schema and schema["required"]:
                    tool_schema["function"]["parameters"]["required"] = schema["required"]

    openai_tools.append(tool_schema)
```

What this does:

- Converts MCP tools to OpenAI function calling format
- Adds permission and risk level to tool descriptions
- LLM can see security context for each tool

- Enables informed decision-making by the LLM

## Step 2b: Get Available Tools - Synthetic Resource Tools

Add synthetic tools for MCP resources:



```
# Add synthetic tools for resources
openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_list_resources",
        "description": "List all available resources from the MCP",
        "parameters": {
            "type": "object",
            "properties": {}
        }
    }
})

openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_read_resource",
        "description": "Read a specific resource by URI from the M",
        "parameters": {
            "type": "object",
            "properties": {
                "uri": {
                    "type": "string",
                    "description": "The URI of the resource to rea"
                }
            },
            "required": ["uri"]
        }
    }
})
```

What this does:

- Creates helper tools for resource access
- Allows LLM to discover and read resources
- Useful for accessing audit logs and configuration
- No permission checks needed (read-only)

## Step 2c: Get Available Tools - Synthetic Prompt Tools

Add synthetic tools for MCP prompts:

```
<> python 
```

```
# Add synthetic tools for prompts
openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_list_prompts",
        "description": "List all available prompt templates from t
        "parameters": {
            "type": "object",
            "properties": {}
        }
    }
})

openai_tools.append({
    "type": "function",
    "function": {
        "name": "mcp_get_prompt",
        "description": "Get a rendered prompt template from the MC
        "parameters": {
            "type": "object",
            "properties": {
                "name": {
                    "type": "string",
                    "description": "The name of the prompt templat
                },
                "arguments": {
                    "type": "object",
                    "description": "Arguments for the prompt templ
                }
            },
            "required": ["name"]
        }
    }
})

return openai_tools
```

What this does:

- Creates helper tools for prompt discovery and usage
- LLM can use security review prompts

- Returns full list of available tools

## Step 3a: Execute Tool - Handle Resource Operations

Create a method to route tool calls with permission enforcement:

```
async def execute_tool(self, tool_name: str, arguments: dict):
    """Execute a tool call with permission checking (real MCP tool or
    await self.connect())

    # Handle synthetic resource tools
    if tool_name == "mcp_list_resources":
        resources = await self.list_resources()
        result = "Available resources:\n"
        for resource in resources:
            result += f"- {resource.uri}"
            if resource.name:
                result += f" ({resource.name})"
            if resource.description:
                result += f": {resource.description}"
            result += "\n"
        return result

    if tool_name == "mcp_read_resource":
        uri = arguments.get("uri")
        if not uri:
            return "Error: URI is required"
        try:
            contents = await self.read_resource(uri)
            if isinstance(contents, list) and len(contents) > 0:
                content = contents[0]
                if hasattr(content, 'text'):
                    return content.text
                return str(content)
            return str(contents)
        except Exception as e:
            return f"Error reading resource: {str(e)}"
```

What this does:

- Routes synthetic resource tools to MCP methods
- Formats resource lists for LLM readability
- Extracts text content from resources



## Step 3b: Execute Tool - Handle Prompt Operations

Add handling for synthetic prompt tools:

```

# Handle synthetic prompt tools
if tool_name == "mcp_list_prompts":
    prompts = await self.list_prompts()
    result = "Available prompts:\n"
    for prompt in prompts:
        result += f"- {prompt.name}"
        if prompt.description:
            result += f": {prompt.description}"
        if hasattr(prompt, 'arguments') and prompt.arguments:
            args = [arg.name for arg in prompt.arguments]
            result += f" (args: {' '.join(args)})"
        result += "\n"
    return result

if tool_name == "mcp_get_prompt":
    name = arguments.get("name")
    prompt_args = arguments.get("arguments", {})
    if not name:
        return "Error: Prompt name is required"
    try:
        messages = await self.get_prompt(name, prompt_args)
        result = f"Prompt: {name}\n\n"
        for msg in messages:
            role = getattr(msg, 'role', 'unknown')
            content = getattr(msg, 'content', '')
            if hasattr(content, 'text'):
                content = content.text
            result += f"[{role}]: {content}\n\n"
        return result
    except Exception as e:
        return f"Error getting prompt: {str(e)}"
```

What this does:

- Routes synthetic prompt tools to MCP methods
- Shows available prompts with their arguments
- Renders prompt templates with substitution
- Returns structured output for LLM consumption

## Step 3c: Execute Tool - Handle Real MCP Tools with Permissions

Add handling for real MCP tools with permission enforcement:

```
<> python 

# Handle regular MCP tools with permission checking
try:
    # Use inherited permission-aware tool execution
    result = await self.call_tool_with_permission(tool_name, argur

# Extract text content from result
if isinstance(result, list) and len(result) > 0:
    content = result[0]
    if hasattr(content, 'text'):
        result_text = content.text
        # Check if this is an approval request
        if "Permission required for tool:" in result_text and
            # Store pending approval
            self.pending_approval = {
                "tool_name": tool_name,
                "arguments": arguments
            }
        return result_text
    return str(content)

if hasattr(result, 'text'):
    return result.text

return str(result)

except Exception as e:
    return f"Error executing tool: {str(e)}"
```

What this does:

- Uses inherited `call_tool_with_permission` method
- Automatically enforces permission policies
- Detects approval request messages
- Stores pending approval details for later execution
- Logs all operations to audit trail
- Extracts and returns text content
- Provides error messages on failure

## Step 4: Implement Risk Assessment

Add a method to assess operation risk:

```
def assess_risk(self, tool_name: str, arguments: dict) -> dict:
    """Assess the risk level of a tool operation."""
    risk_level = self.risk_levels.get(tool_name, "medium")
    permission = self.permissions.get(tool_name, "ask")

    assessment = {
        "tool": tool_name,
        "risk_level": risk_level,
        "permission": permission,
        "requires_approval": permission in ["ask", "deny"],
        "description": ""
    }

    # Add risk-specific descriptions
    if risk_level == "low":
        assessment["description"] = "Safe operation with minimal impact"
    elif risk_level == "medium":
        assessment["description"] = "Moderate impact - modifies data"
    elif risk_level == "high":
        assessment["description"] = "High impact - destructive operation"
    elif risk_level == "critical":
        assessment["description"] = "Critical impact - system-level operation"

    return assessment
```

What this does:

- Determines risk level for each tool
- Checks current permission policy
- Returns structured risk assessment
- Provides human-readable descriptions
- Helps LLM and users understand implications

## Step 5: Implement the Chat Method

Create the main chat method with permission-aware tool calling:



```

self.pending_approval = None # Clear pending approval
response_text = "Operation cancelled by user."

self.conversation_history.append({
    "role": "user",
    "content": user_message
})
self.conversation_history.append({
    "role": "assistant",
    "content": response_text
})

return response_text

# Add user message to conversation history
self.conversation_history.append({
    "role": "user",
    "content": user_message
})

# Get available tools
tools = await self.get_available_tools()

# Call OpenAI with tools (only pass tools if they exist)
if tools:
    response = self.llm_client.chat.completions.create(
        model=self.model,
        messages=self.conversation_history,
        tools=tools,
        tool_choice="auto"
    )
else:
    response = self.llm_client.chat.completions.create(
        model=self.model,
        messages=self.conversation_history
    )

if not response or not response.choices:
    return "Error: No response from LLM"

assistant_message = response.choices[0].message

# Handle tool calls
if assistant_message.tool_calls:
    # Add assistant's message with tool calls to history
    self.conversation_history.append({
        "role": "assistant",
        "content": assistant_message.content or "",
        "tool_calls": [
            {
                "id": tc.id,

```

```

        "name": tc.function.name,
        "arguments": tc.function.arguments
    }
}
for tc in assistant_message.tool_calls
]
}))

# Execute each tool call with permission checking
for tool_call in assistant_message.tool_calls:
    function_name = tool_call.function.name
    function_args = json.loads(tool_call.function.arguments)

    # Assess risk before execution
    risk_assessment = self.assess_risk(function_name, function

    # Execute the tool (permission checking happens inside)
    tool_result = await self.execute_tool(function_name, funct

    # Add tool result to history
    self.conversation_history.append({
        "role": "tool",
        "tool_call_id": tool_call.id,
        "content": str(tool_result)
    })

# Get final response after tool execution
final_response = self.llm_client.chat.completions.create(
    model=self.model,
    messages=self.conversation_history
)

if not final_response or not final_response.choices:
    return "Error: No response from LLM after tool execution"

final_message = final_response.choices[0].message.content
self.conversation_history.append({
    "role": "assistant",
    "content": final_message
})

return final_message

else:
    # No tool calls, just return the response
    self.conversation_history.append({
        "role": "assistant",
        "content": assistant_message.content
    })

    return assistant_message.content

```

What this does:

- Detects approval/denial responses for pending operations
- Executes approved tools with `approved=True` flag
- Handles user cancellations of pending operations
- Manages multi-turn LLM conversations
- Provides tools with permission metadata
- Assesses risk before each operation
- Enforces permissions automatically
- Logs all operations to audit trail
- Returns final response after tool execution

## Step 6: Create the Gradio Interface

Build a chat interface with permission awareness:



python



```
def create_interface(self):
    """Create the Gradio chat interface with permission awareness."""

    async def chat_wrapper(message, history):
        """Wrapper for chat method compatible with Gradio."""
        if not message.strip():
            return history

        response = await self.chat(message, history)
        # Return updated history with new messages
        return history + [
            {"role": "user", "content": message},
            {"role": "assistant", "content": response}
        ]

    async def reset_conversation():
        """Reset the conversation history."""
        self.conversation_history = []
        return []

    with gr.Blocks(title="MCP Permission AI Host") as interface:
        gr.Markdown(f"""
            # MCP Permission AI Host
            Chat with GPT-4o-mini using permission-aware MCP tools.

            **Model:** {self.model}
            **Permissions:** Enforced with audit logging
            **Risk Assessment:** Automatic for all operations

            All tool executions are subject to permission policies and log
            """)

        chatbot = gr.Chatbot(
            label="Conversation",
            height=500,
            type="messages"
        )

        with gr.Row():
            msg = gr.Textbox(
                label="Your message",
                placeholder="Ask me to use MCP tools...",
                scale=4
            )
            clear = gr.Button("Clear", scale=1)

        with gr.Accordion("Permission Status", open=False):
            perm_info = gr.Markdown(self._get_permission_summary())
```



```

        fn=chat_wrapper,
        inputs=[msg, chatbot],
        outputs=chatbot
    ).then(
        lambda: "",
        outputs=msg
    )

    clear.click(
        fn=reset_conversation,
        outputs=chatbot
    )

    return interface

def _get_permission_summary(self) -> str:
    """Generate a summary of current permissions."""
    summary = "### Current Permission Policies:\n\n"
    for tool, policy in self.permissions.items():
        risk = self.risk_levels.get(tool, "medium")
        summary += f"- **{tool}**: {policy.upper()} (Risk: {risk})\n"
    return summary

```

What this does:

- Creates chat interface with permission context
- Shows permission status in accordion
- Displays risk levels for each tool
- Provides clear/reset functionality
- All operations automatically logged

## Step 7: Add Main Entry Point

Add the main function to run the AI host:



python



```
def main():
    if len(sys.argv) < 2:
        print("Usage: python mcp_permission_host_app.py <server_script>")
        print("Example: python mcp_permission_host_app.py mcp_permission_s
        sys.exit(1)

    server_script = sys.argv[1]

    client = MCPPPermissionHostApp(server_script)
    interface = client.create_interface()
    interface.queue().launch(server_name="127.0.0.1", server_port=7864)

if __name__ == "__main__":
    main()
```

What this does:

- Accepts server script path as argument
- Creates permission-aware AI host
- Launches on port 7864 (distinct from other apps)
- Uses `.queue()` for async support

Click below to see the complete code for `mcp_permission_host_app.py`:

► Complete code for `mcp_permission_host_app.py` (click to expand)

# Test the GUI Client App

Now that we have built all components, let's test the complete permission and audit system.

## Create Test Data

First, create a sample file for testing:

bash

```
cd mcp_security_lab
echo "Sample content for testing" > data/test.txt
```

Run

## Run the GUI Client App

Open a terminal and run the GUI client:

bash

```
cd mcp_security_lab
source ../mcp_security_env/bin/activate
python mcp_permission_client_app.py mcp_permission_server.py
```

Run

The script creates an application with a user interface built by Gradio. The application is ready to launch when you see output similar to the following in your terminal:

plaintext

```
Running on local URL: http://127.0.0.1:7863
```

This message indicates that the application is ready for launch and is being served

below:

GUI Client

**Note:** if this **GUI Client** button above does not work, follow these instructions to launch the application:

- On the menu bar at the top, select **Launch Application**
- Enter port **7863**
- Click **OK**

## Test 1: Permission Enforcement with Allow Policy

Let's test that tools with "allow" permission execute automatically without prompting.

### Steps:

1. In the GUI client app, navigate to the **Permissions** tab
2. Click **Load Tools** to populate the dropdown with available tools
3. Configure permissions:
  - Select **read\_file** from the dropdown
  - Select the **Allow** radio button
  - Click **Save Permission**
4. Navigate to the **Tools** tab
5. Call the **read\_file** tool:
  - Tool: **read\_file**
  - Arguments: **{"filepath": "test.txt"}**
  - Click **Call Tool**

### Expected Result:

The tool executes immediately without any approval prompt. You should see:



plaintext



```
Sample content for testing
```

The audit log in the **Permissions** tab should show:



plaintext



```
[timestamp] ALLOWED: read_file with args {'filepath': 'test.txt'}
```

## Test 2: Permission Enforcement with Deny Policy

Let's test that tools with "deny" permission are blocked automatically.

### Steps:

1. In the **Permissions** tab, click **Load Tools** if not already loaded
2. Configure the deny permission:
  - Select **delete\_file** from the dropdown
  - Select the **Deny** radio button
  - Click **Save Permission**
3. Navigate to the **Tools** tab
4. Attempt to call the **delete\_file** tool:
  - Tool: **delete\_file**
  - Arguments: **{"filepath": "test.txt"}**
  - Click **Call Tool**

### Expected Result:

The tool is blocked immediately. You should see:



plaintext



```
Permission denied for tool: delete_file
```

The audit log should show:



plaintext



```
[timestamp] DENIED: delete_file with args {'filepath': 'test.txt'}
```

## Test 3: Permission Enforcement with Ask Policy

Let's test that tools with "ask" permission prompt for user approval.

### Steps:

1. In the **Permissions** tab, click **Load Tools** if not already loaded
2. Configure the ask permission:
  - Select **write\_file** from the dropdown
  - Select the **Ask** radio button
  - Click **Save Permission**
3. Navigate to the **Tools** tab
4. Call the **write\_file** tool:
  - Tool: **write\_file**
  - Arguments: **{"filepath": "new\_file.txt", "content": "Test content"}**
  - Click **Call Tool**

### Expected Result:

You should see a permission request prompt:

plaintext

```
Permission required for tool: write_file
Arguments: {
  "content": "Test content",
  "filepath": "new_file.txt"
}

This tool requires approval before execution.
Please approve this operation in the GUI to proceed.
```

5. Now click the **Approve & Execute** button to approve and execute the tool.

### Expected Result after approval:

The tool executes successfully:

plaintext

```
Successfully wrote to new_file.txt
```

The audit log should show:

plaintext

```
[timestamp] ASK: write_file - Decision: Awaiting approval
[timestamp] ALLOWED: write_file - Decision: Policy: ask
```

# Test the AI Host App

Now let's test the AI host's risk assessment and permission enforcement with the LLM-powered interface.

## Run the AI Host App

If the GUI client is still running, you can either:

- Stop it (Ctrl+C) and use the same terminal
- Or open another **new terminal** and activate the environment:

 bash 

```
cd mcp_security_lab && source ../mcp_security_env/bin/activate
```



Run the AI host application:

 bash 

```
python mcp_permission_host_app.py mcp_permission_server.py
```



The script creates an application with a chat interface built by Gradio. The application is ready to launch when you see output similar to the following in your terminal:

 plaintext 

```
Running on local URL: http://127.0.0.1:7864
```

This message indicates that the application is ready for launch and is being served on port **7864**. You can launch the app now by clicking on the **Host Application** button below:



**Note:** if this **Host Application** button above does not work, follow these instructions to launch the application:

- On the menu bar at the top, select **Launch Application**
- Enter port **7864**
- Click **OK**

## Test 4: AI Host with Risk Assessment

Let's test the AI host's risk assessment and permission enforcement.

### Steps:

1. Check the **Permission Status** accordion to see current policies
2. Ask the AI to perform a low-risk operation:

 plaintext 

Read the contents of test.txt

### Expected Result:

Since **read\_file** has "allow" permission and LOW risk, the AI will execute it immediately and return the file contents.

3. Ask the AI to perform a high-risk operation:

 plaintext 

Delete the file test.txt

### Expected Result:

Since **delete\_file** has "deny" permission and HIGH risk, the AI will be blocked by the permission policy and will inform you that the operation was denied.

4. Ask the AI to perform a medium-risk operation with "ask" permission:



plaintext



Write a new file called `greeting.txt` with the content "Hello, world!"

**Note:** LLM behavior can be non-deterministic. Sometimes the AI might not call the tool on the first attempt, even though it's available. If the AI responds without calling the tool or says it cannot perform the action, simply try asking again with the same or a rephrased request. You might need to try several times until the LLM successfully calls the `write_file` tool.

### Expected Result:

The AI will attempt to call the `write_file` tool, but since it has "ask" permission, it will show a permission request message:



plaintext



```
Permission required for tool: write_file
Arguments: {
  "filepath": "greeting.txt",
  "content": "Hello, world!"
}
```

This tool requires approval before execution.  
Please approve this operation in the GUI to proceed.

5. Type "yes" to approve the operation

### Expected Result after approval:

The system will execute the approved operation and respond with:



plaintext



```
Operation approved and executed.

Successfully wrote to greeting.txt
```

The operation is now complete and logged in the audit trail.

# Deployment Summary

You now have a complete MCP security system with:

1. **Server** ( `mcp_permission_server.py` ): 4 risk-tiered tools with audit logging
2. **Base Client** ( `mcp_permission_client_base.py` ): Permission checking and audit framework
3. **GUI Client** ( `mcp_permission_client_app.py` ): Permission management interface (port 7863)
4. **AI Host** ( `mcp_permission_host_app.py` ): Risk-aware AI assistant (port 7864)

## Key Features Demonstrated:

- **Permission Policies:** Allow, deny, and ask enforcement
- **Risk Assessment:** Four-tier risk classification (low, medium, high, critical)
- **Audit Logging:** Complete operation tracking with timestamps
- **Argument-Specific Permissions:** Granular control over specific operations
- **Human-in-the-Loop:** User approval for sensitive operations
- **AI Integration:** Permission-aware tool calling with risk display
- **Resources:** Access to audit logs and configuration
- **Prompts:** Security review templates

Your permission system is production-ready and can be extended with additional tools, custom risk assessment logic, and more sophisticated elicitation patterns.

# Conclusion

Congratulations! You have successfully built a production-grade MCP security system with comprehensive permission management, risk assessment, and audit logging.

## What You Accomplished

In this lab, you created four interconnected components that work together to provide enterprise-level security:

### 1. **Permission-Aware MCP Server** ( `mcp_permission_server.py` )

- Four risk-tiered tools (LOW, MEDIUM, HIGH, CRITICAL)
- Automatic audit logging for all operations
- Resources exposing audit logs and configuration
- Security review prompts for operation analysis

### 2. **Base Permission Client** ( `mcp_permission_client_base.py` )

- Three-tier permission policy system (allow, deny, ask)
- Argument-specific permission overrides
- Persistent permission storage (JSON-based)
- Complete audit trail with timestamps
- Protocol methods for tools, resources, and prompts

### 3. **GUI Client Application** ( `mcp_permission_client_app.py` )

- User-friendly permission management interface
- Four-tab organization (Tools, Resources, Prompts, Permissions)
- Real-time permission status display
- Audit log viewer with scrollable history
- Visual permission configuration with radio buttons

- OpenAI GPT-4o-mini integration for intelligent assistance
- Automatic risk assessment for all operations
- Permission-aware tool calling with human-in-the-loop
- Risk level display in tool descriptions
- Permission status accordion in chat interface

## Key MCP Security Concepts Mastered

### 1. Permission Management

You learned how to implement a comprehensive permission system:

```
<> python   
  
# Three-tier policy enforcement  
permissions = {  
    "read_file": "allow",      # Auto-execute  
    "write_file": "ask",       # Prompt user  
    "delete_file": "deny"     # Block completely  
}  
  
# Argument-specific overrides for granular control  
"write_file:{\"filepath\": \"safe.txt\"}": "allow"
```

**Key Insight:** Permission policies should be context-aware, supporting both tool-level and argument-level control for maximum flexibility.

### 2. Risk Assessment

You implemented a four-tier risk classification system:

- **LOW:** Read operations, safe queries
- **MEDIUM:** Write operations, data modifications
- **HIGH:** Delete operations, destructive actions
- **CRITICAL:** Command execution, system-level operations

**Key Insight:** Risk levels help users make informed decisions about which operations to approve, improving security without sacrificing usability.

### 3. Audit Logging

You created a complete audit trail tracking:

- Permission decisions (ALLOWED, DENIED, ASKED)
- User approval events
- Tool execution outcomes
- Timestamps for compliance and debugging
- Full argument capture for forensics

**Key Insight:** Comprehensive logging is essential for security compliance, debugging, and understanding system behavior over time.

### 4. Elicitation (Conceptual)

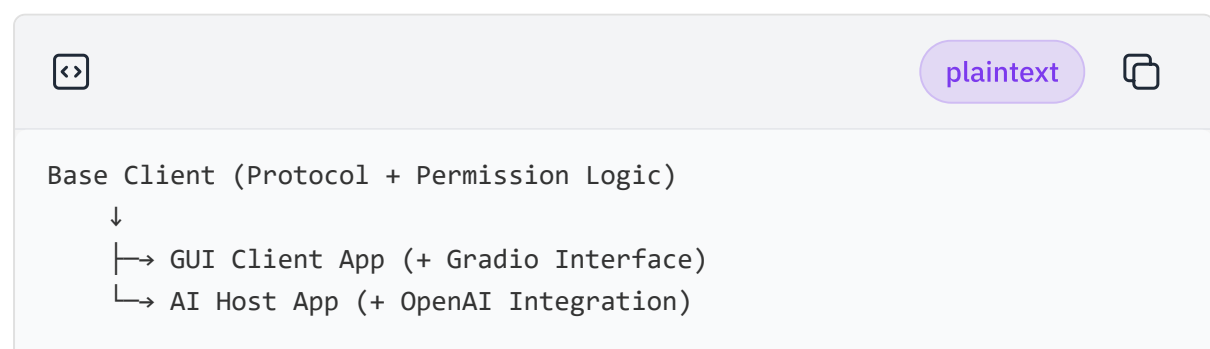
Although this lab included a conceptual elicitation implementation, you learned the foundations:

- Structured input collection via JSON schemas
- Three-action model (Accept, Decline, Cancel)
- Validation before sensitive operations
- User-friendly prompts with context

**Key Insight:** Elicitation enables interactive workflows where tools can request missing information or confirmation from users.

## Architecture Patterns

This lab demonstrates a clean architecture pattern with proper separation of concerns:



1. **Code Reuse:** Protocol logic written once, shared by all applications
2. **Separation of Concerns:** Each class has a single, well-defined responsibility
3. **Maintainability:** Changes to protocol logic automatically propagate to all apps
4. **Testability:** Base client can be tested independently of UI/AI layers
5. **Extensibility:** New applications can easily inherit from base client

## Architecture Benefits

### Security at Every Layer:

- Permission policies enforce access control
- Risk assessment informs user decisions
- Audit logs provide accountability
- Fail-safe defaults protect against mistakes

### Flexibility and Control:

- Three-tier permission model (allow/deny/ask)
- Argument-specific overrides for granular control
- Dynamic risk assessment based on context
- Human-in-the-loop for sensitive operations

### Production Ready:

- Complete audit trail for compliance
- Persistent permission storage
- Integration-friendly design
- Extensible risk assessment framework

### Code Reusability:

- Base client provides protocol and permission logic
- Both applications inherit seamlessly
- No code duplication between GUI and AI host
- Easy to add new permission-aware applications

# Key Patterns

## Three-Tier Permission System:

```
permissions = {
    "read_file": "allow",      # Auto-execute
    "write_file": "ask",       # Prompt user
    "delete_file": "deny"     # Block completely
}

# Check permission before execution
permission = check_permission(tool_name, arguments)
if permission == "allow":
    execute()
elif permission == "ask":
    if user_approves():
        execute()
else: # deny
    log_and_block()
```

## Risk Assessment:

```
risk_levels = {
    "read_file": "low",
    "write_file": "medium",
    "delete_file": "high",
    "execute_command": "critical"
}

# Communicate risk to users
risk = assess_risk(tool_name, arguments)
display(f"Risk: {risk['level']}")
```

## Audit Logging:

```
# Log every decision with timestamp
log_entry = f"[{timestamp}] {decision}: {tool_name}"
log_entry += f" with args {arguments}\n"
```



## Argument-Specific Permissions:



python



```
# General deny, but allow specific safe cases
"write_file": "deny",
"write_file:{\"filepath\": \"safe.txt\"}": "allow"
```

## What's Next?

Now you can:

1. Add more risk-tiered tools and test permission enforcement
2. Implement custom risk assessment logic for your domain
3. Create new permission-aware apps (CLI, API) by inheriting from base
4. Apply permission patterns to your own MCP servers and clients

You now have a solid foundation for building secure, production-grade MCP applications with comprehensive permission management!

## Author(s)

[Wojciech \"Victor\" Fulmyk](#)

© IBM Corporation. All rights reserved.

# MCP Security with Permissions and Elicitation

Estimated time needed: 60 minutes

In this lab, you'll build an advanced MCP security system featuring **permissions** for policy-based access control and **elicitation** for structured user input. You'll create a **base permission-aware client** that enforces security policies, then build two applications on top of it: a **GUI client app** with permission management interface and an **AI-powered host app** that assesses risk and requests user approval for sensitive operations.

This design demonstrates enterprise-grade security patterns for production MCP deployments with comprehensive audit logging.

## Learning Objectives

After completing this lab, you will be able to:

- Implement permission policies (allow, deny, ask) for tool execution
- Configure argument-specific permission rules for granular control
- Use elicitation to request structured user input with JSON schemas
- Build audit logging systems for compliance and security monitoring
- Assess operation risk levels and apply appropriate controls
- Create interactive approval workflows for sensitive operations
- Manage permissions programmatically and through GUI interfaces

## Prerequisites

Before starting this lab, you should have:

- Experience building MCP clients with base/derived class architecture
- Understanding of MCP protocol methods (tools, resources, prompts)
- Familiarity with async/await patterns in Python
- Basic knowledge of object-oriented programming (inheritance)
- Understanding of access control and security policies
- Familiarity with JSON Schema validation
- Awareness of audit logging and compliance requirements

---

Lab Setup →